

Recetario Fork

Aplicacion web con versiones tipo fork

Documentacion de la practica

Asignatura: ALS - Cuarto curso

Tecnologias: Flask / Jinja2 / Sirope-Redis / Flask-Login / HTMX

Python 3.11+

Indice

1. Introduccion y objetivos
2. Requisitos y tecnologias
3. Arquitectura de la aplicacion
4. Modelo de datos (Entidades)
5. Capa de persistencia - Sirope / Redis
6. Vistas y rutas (Blueprints)
7. Plantillas y frontend
8. Funcionalidades principales
9. Diagramas
10. Estructura de ficheros
11. Instalacion y ejecucion
12. Decisiones de diseno

1. Introducción y objetivos

<<Recetario Fork>> es una aplicación web desarrollada con Flask que permite a los usuarios publicar recetas de cocina y, de forma análoga al concepto de fork en sistemas de control de versiones, crear variantes personalizadas de recetas existentes. Cada receta forkeada mantiene la referencia a su receta padre y a la raíz del árbol de versiones, permitiendo navegar por el historial de modificaciones y comparar diferencias entre versiones.

Los objetivos principales de la práctica son:

- Diseñar e implementar una aplicación web completa con al menos 4 entidades de dominio (sin contar User).
- Utilizar Sirope como ORM sobre Redis para la persistencia de datos.
- Implementar autenticación y autorización de usuarios con Flask-Login.
- Aplicar el patrón application factory de Flask con blueprints para organizar el código.
- Ofrecer interactividad mejorada con HTMX para operaciones sin recarga de página.
- Demostrar el uso de relaciones entre entidades (árbol de forks, comentarios, valoraciones, favoritos, notificaciones).

2. Requisitos y tecnologías

2.1 Requisitos del sistema

- Python 3.11 o superior.
- Servidor Redis corriendo en local (por defecto localhost:6379).
- Navegador web moderno con JavaScript habilitado (para HTMX).

2.2 Dependencias Python

Las dependencias se definen en requirements.txt:

```
Flask>=3.0
Flask-Login>=0.6
Flask-WTF>=1.2
WTForms>=3.1
email-validator>=2.0
sirope>=0.3
redis>=5.0
```

2.3 Stack tecnológico

Capa	Tecnología
Backend	Flask 3.x (Python)
Plantillas	Jinja2 (integrado en Flask)
ORM / Persistencia	Sirope sobre Redis
Autenticación	Flask-Login + SHA-256
Formularios	Flask-WTF / WTForms
Frontend interactivo	HTMX 1.9 (comentarios, valoraciones)
CSS	Pico CSS 2.x + custom style.css

3. Arquitectura de la aplicación

La aplicación sigue el patrón Application Factory de Flask. La función `create_app()` en `app/__init__.py` construye y configura la instancia de Flask, registra las extensiones (LoginManager), importa y registra los blueprints, inyecta variables globales en las plantillas (como el contador de notificaciones no leídas) y registra los manejadores de errores HTTP.

3.1 Patrón Application Factory

Este patrón permite crear múltiples instancias de la aplicación con distintas configuraciones (test, desarrollo, producción). La configuración se carga desde `config.py` vía `from_object()`. El punto de entrada es `run.py`, que simplemente invoca `create_app()` y arranca el servidor de desarrollo.

3.2 Blueprints

Las rutas se organizan en 6 blueprints independientes, cada uno en su propio módulo dentro de `app/views/`:

Blueprint	Prefijo	Responsabilidad
auth	/auth	Login, registro, logout
main	/	Índice, búsqueda, filtrado
recipes	/recipes	CRUD recetas, fork, diff, tree, export
users	/user	Perfil público del usuario
favorites	/favorites	Listado de favoritos
notifications	/notifications	Bandeja de notificaciones

Arquitectura del Sistema - Recetario Fork

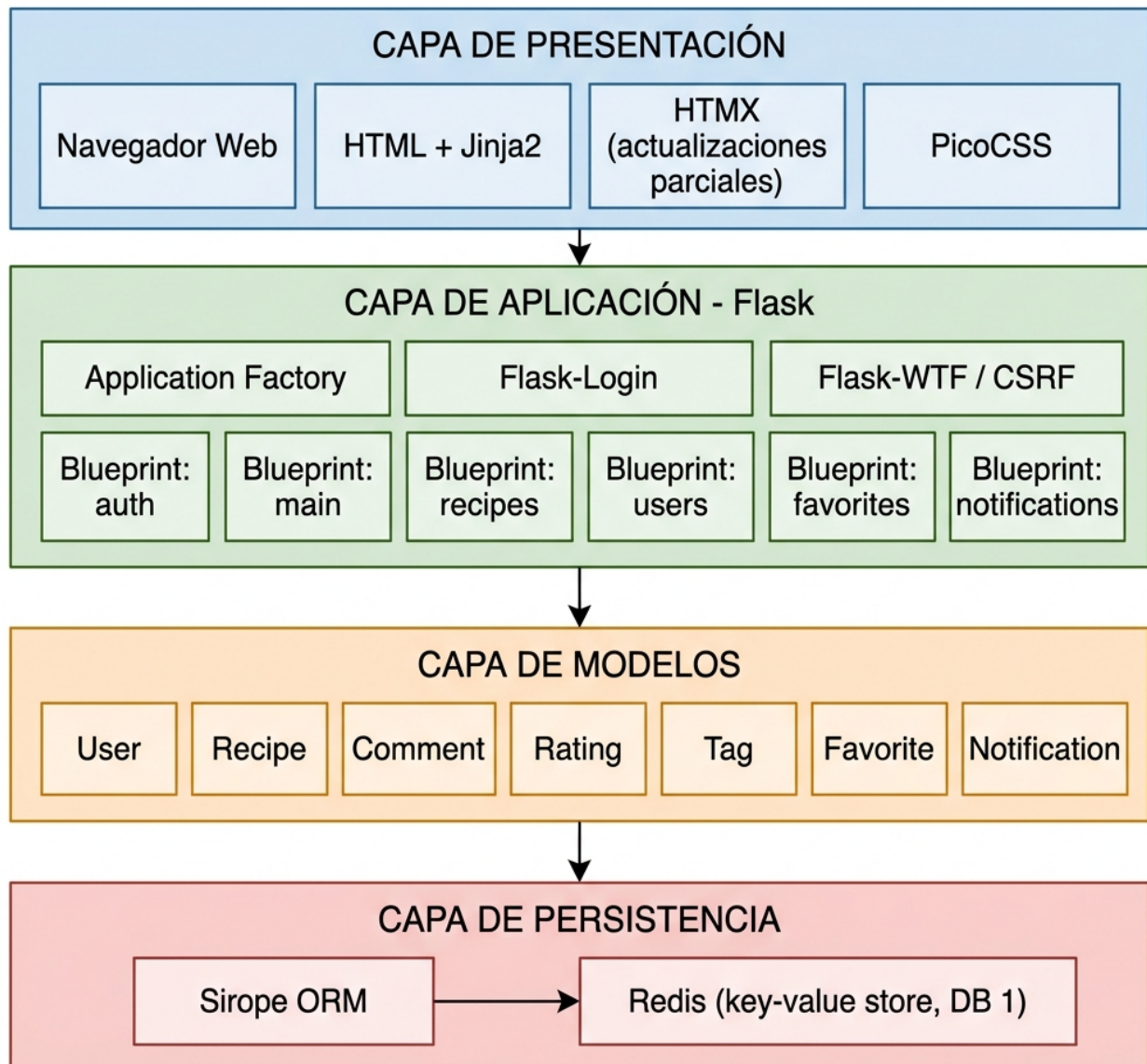


Figura 1 - Diagrama de arquitectura de la aplicación

4. Modelo de datos (Entidades)

El dominio se compone de 7 clases Python planas (sin herencia de un ORM tradicional). Sirope serializa sus atributos directamente en Redis. Se distinguen 6 entidades de dominio más la clase User (que hereda de UserMixin de Flask-Login).

4.1 User

Representa un usuario registrado. Integra Flask-Login mediante UserMixin. La contraseña nunca se almacena en claro: se guarda su hash SHA-256. La identidad es el username (único), que también se devuelve en get_id().

- **Atributos:** _username (str), _email (str), _pwd_hash (str)
- **Métodos clave:** check_password(pwd), set_password(pwd), get_id()

4.2 Recipe

Entidad central. Almacena título, descripción, ingredientes (lista), pasos (lista), etiquetas, imagen (URL o ruta local) y la información de linaje para el sistema de forks.

- **parent_oid:** OID seguro de la receta padre (None si es original)
- **root_oid:** OID seguro de la raíz del árbol de forks
- **Método is_fork():** True si parent_oid no es None
- **Método diff_against(other):** Calcula ingredientes añadidos/eliminados y si los pasos cambiaron

4.3 Comment

Comentario que un usuario deja sobre una receta. Almacena recipe_oid, author_username, text y created_at (timestamp ISO).

4.4 Rating

Valoración de 1 a 5 estrellas. La unicidad (un rating por usuario y receta) se garantiza en la capa de vistas: al votar, si ya existe un Rating del usuario, se actualizan las estrellas en lugar de crear uno nuevo.

- **Atributos:** _recipe_oid, _author_username, _stars (recortado a [1,5])

4.5 Tag

Etiqueta (categoría libre) asociada a recetas. El nombre se normaliza a minúsculas y sin espacios sobrantes para evitar duplicados.

4.6 Favorite

Marca de favorito de un usuario sobre una receta. Funciona como toggle: al pulsar el botón, si ya existe se elimina; si no, se crea.

4.7 Notification

Aviso dirigido a un usuario cuando alguien interactúa con su receta. Tipos soportados: 'fork' (otro usuario forkeó tu receta) y 'comment' (otro usuario comentó en tu receta). Tiene un flag _read para marcar como leída.

Recetario Fork

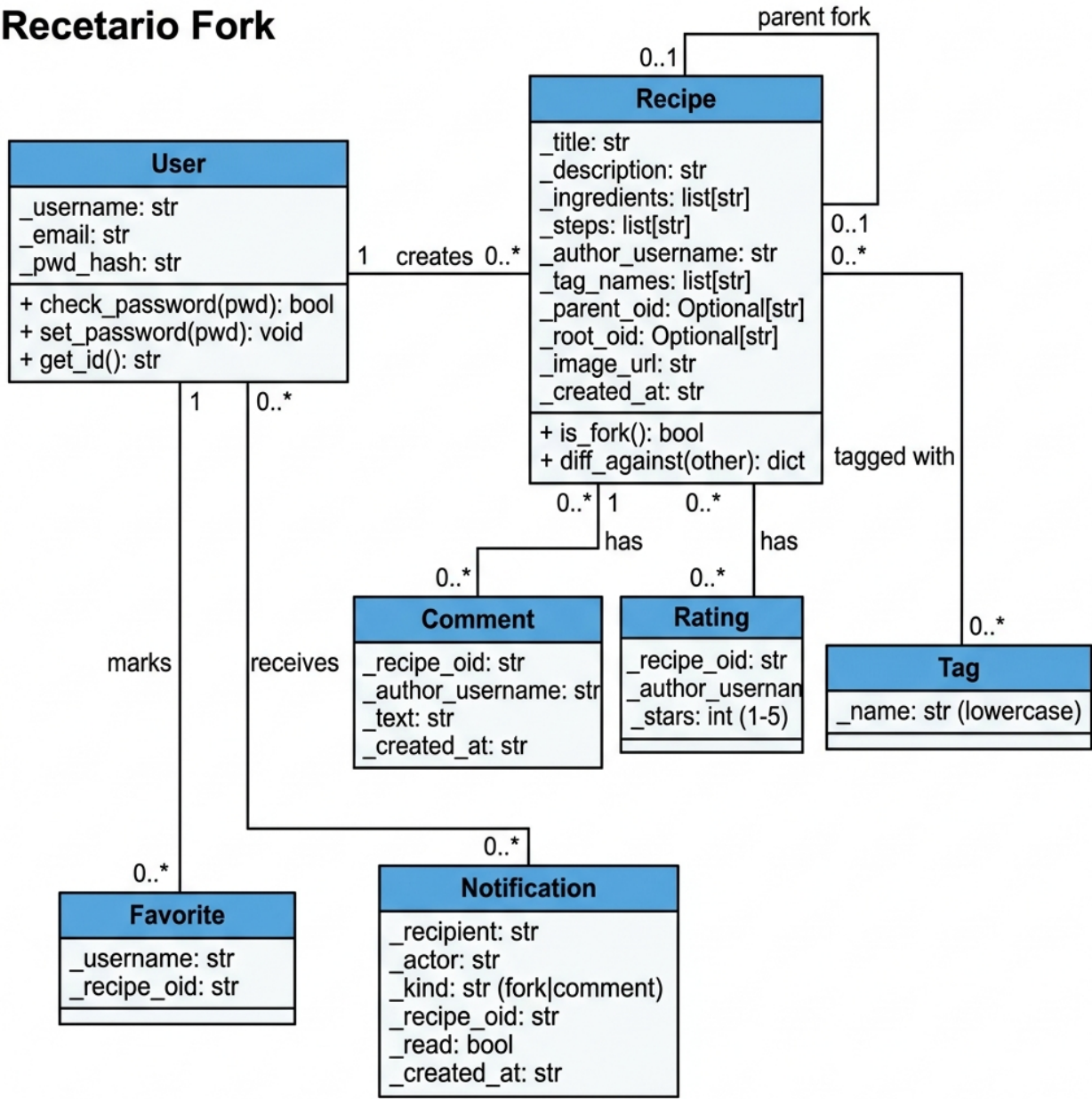


Figura 2 - Diagrama de clases del modelo de datos

5. Capa de persistencia - Sirope / Redis

Sirope es una librería ligera de persistencia de objetos Python sobre Redis. Los objetos se serializan con pickle y se almacenan como valores binarios en Redis. Cada objeto guardado recibe un OID (Object Identifier) único asignado por Sirope.

5.1 Configuración

En `app/extensions.py` se crea el cliente Redis y la instancia global de Sirope (`srp`). La conexión se configura mediante variables de entorno opcionales: `REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_DB` (por defecto DB 1).

```
_redis_client = redis.Redis(  
    host=os.environ.get('REDIS_HOST', 'localhost'),  
    port=int(os.environ.get('REDIS_PORT', 6379)),  
    ...  
)  
srp = sirope.Sirope(_redis_client)
```

5.2 Operaciones principales

- **srp.save(obj):** Guarda o actualiza un objeto. Devuelve el OID
- **srp.load(oid):** Carga un objeto por su OID
- **srp.load_all(Class):** Iterador sobre todos los objetos de una clase
- **srp.delete(oid):** Elimina un objeto de Redis
- **srp.exists(oid):** Comprueba si un OID sigue existiendo

5.3 OIDs seguros para URLs

Los OIDs internos de Sirope no son directamente seguros para pasar por URLs. Se utilizan dos funciones helper definidas en `extensions.py`:

- **safe_oid(obj):** Convierte `obj.__oid__` a su forma URL-safe (cadena)
- **load_safe(safe):** Reconstruye el OID desde la cadena segura y carga el objeto (o None)

6. Vistas y rutas (Blueprints)

6.1 Blueprint auth (/auth)

Gestión de autenticación con Flask-Login.

- **GET/POST /auth/login:** Formulario de inicio de sesión. Respeta ?next= para redirigir al destino previsto
- **GET/POST /auth/register:** Registro de nuevos usuarios. Rechaza usernames duplicados. Tras crear la cuenta inicia sesión automáticamente
- **GET /auth/logout:** Cierra la sesión y redirige al índice

6.2 Blueprint main (/)

Página principal con listado de recetas y filtros.

- **GET /:** Listado de recetas con filtros opcionales: ?q= (búsqueda texto), ?tag= (etiqueta), ?author= (usuario)

6.3 Blueprint recipes (/recipes)

CRUD completo de recetas, forks, árbol de versiones, diff y exportación.

- **GET/POST /recipes/new:** Crear receta original
- **GET /recipes/<safe>:** Detalle completo de la receta con comentarios, valoraciones, forks hijos
- **GET/POST /recipes/<safe>/edit:** Editar receta (solo el autor)
- **POST /recipes/<safe>/delete:** Borrar receta con política de huérfanos para los forks hijos
- **POST /recipes/<safe>/fork:** Crear un fork de la receta
- **GET /recipes/<safe>/tree:** Árbol completo de versiones (recursivo)
- **GET /recipes/<safe>/diff:** Comparación de cambios respecto al padre
- **POST /recipes/<safe>/comment:** Añadir comentario (compatible con HTMX)
- **POST /recipes/<safe>/comment/<csafe>/delete:** Borrar comentario propio
- **POST /recipes/<safe>/rate:** Valorar con estrellas 1-5 (compatible con HTMX)
- **POST /recipes/<safe>/favorite:** Toggle favorito (compatible con HTMX)
- **POST /recipes/<safe>/merge-from-parent:** Traer ingredientes/pasos del padre al fork
- **GET /recipes/<safe>/export:** Descargar la receta en formato Markdown

6.4 Blueprint users (/user)

- **GET /user/<username>:** Perfil público: recetas del usuario, estadísticas (originales, forks, media de valoraciones recibidas)

6.5 Blueprint favorites (/favorites)

- **GET /favorites/:** Listado de recetas marcadas como favoritas por el usuario autenticado

6.6 Blueprint notifications (/notifications)

- **GET /notifications/:** Bandeja de notificaciones. Al visitar se marcan todas como leídas
- **POST /notifications/clear:** Elimina todas las notificaciones del usuario

Mapa de Rutas URL - Recetario Fork

METHOD	URL	BLUEPRINT	DESCRIPCIÓN
GET	/	main	Listado de recetas (con filtros q, tag, author)
GET/POST	/auth/login	auth	Inicio de sesión
GET/POST	/auth/register	auth	Registro de usuario
GET	/auth/logout	auth	Cerrar sesión
GET/POST	/recipes/new	recipes	Crear receta nueva
GET	/recipes/<safe>	recipes	Detalle de receta
GET	/recipes/<safe>	recipes	Editar receta
GET/POST	/recipes/<safe>/edit	recipes	Editar receta
POST	/recipes/<safe>/delete	recipes	Borrar receta
POST	/recipes/<safe>/fork	recipes	Forkear receta
GET	/recipes/<safe>/tree	recipes	Árbol de versiones
GET	/recipes/<safe>/diff	recipes	Diferencias con padre
POST	/recipes/<safe>/comment	recipes	Añadir comentario
POST	/recipes/<safe>/comment/<csafe>/delete	recipes	Borrar comentario
POST	/recipes/<safe>/rate	recipes	Valorar receta
POST	/recipes/<safe>/favorite	recipes	Toggle favorito
POST	/recipes/<safe>/merge-from-parent	recipes	Merge del padre
GET	/recipes/<safe>/export	recipes	Exportar Markdown
GET	/user/<username>	users	Perfil de usuario
GET	/favorites/	favorites	Mis favoritos
GET	/notifications/	notifications	Bandeja de notificaciones
POST	/notifications/clear	notifications	Limpiar notificaciones

Figura 3 - Mapa de rutas URL de la aplicación

7. Plantillas y frontend

7.1 Motor de plantillas - Jinja2

Todas las páginas heredan de base.html, que define la estructura HTML, la barra de navegación (con enlaces dinámicos según si el usuario está autenticado o no), el contenedor de mensajes flash y el bloque content que cada plantilla rellena.

7.2 CSS - Pico CSS + estilos propios

Se utiliza Pico CSS 2.x como framework CSS ligero que proporciona un estilo moderno y responsivo sin clases. Los estilos adicionales (tags, badges, árbol de versiones, diff, etc.) se definen en static/style.css.

7.3 HTMX - Interactividad sin JavaScript

HTMX 1.9 se carga desde CDN en base.html. Se utiliza para tres operaciones que se ejecutan sin recarga completa de página:

- Añadir/borrar comentarios: el formulario envía con hx-post y el servidor devuelve el fragmento `_comments.html`.
- Valorar con estrellas: el formulario envía con hx-post y el servidor devuelve el fragmento `_rating.html`.
- Toggle de favoritos: el botón envía con hx-post y el servidor devuelve el fragmento `_fav_btn.html`.

7.4 Parciales (fragments)

Se utilizan 4 plantillas parciales para respuestas HTMX y para reutilización:

- `_card.html`: Tarjeta resumen de una receta (usada en listados)
- `_comments.html`: Lista de comentarios con botón de borrar
- `_rating.html`: Panel de valoración con estrellas y media
- `_fav_btn.html`: Botón toggle de favorito

8. Funcionalidades principales

8.1 Sistema de forks (versiones)

La funcionalidad central de la aplicación. Cualquier usuario autenticado puede forkear una receta existente. Al hacerlo se crea una copia con `parent_oid` apuntando al original y `root_oid` apuntando a la raíz del árbol. El usuario aterriza en el formulario de edición para personalizar su variante.

8.2 Árbol de versiones

Desde cualquier receta se puede navegar al árbol completo de versiones. La vista construye el árbol de forma recursiva partiendo de la raíz. La receta actual aparece resaltada. Cada nodo muestra título y autor.

8.3 Diff entre versiones

Se pueden comparar los cambios de un fork respecto a su padre. El método `diff_against()` de `Recipe` calcula: ingredientes añadidos, ingredientes eliminados y si los pasos han cambiado (mostrando ambas listas lado a lado).

8.4 Merge desde el padre

El autor de un fork puede traer los ingredientes y pasos actuales del padre (sobreescribiendo los propios). Requiere confirmación del usuario antes de ejecutarse.

8.5 Comentarios y valoraciones

Los usuarios autenticados pueden dejar comentarios (texto libre, máx. 500 caracteres) y valoraciones de 1 a 5 estrellas en cualquier receta. Ambas operaciones son compatibles con HTMX para actualizarse sin recargar la página. La unicidad de valoraciones se garantiza en la capa de vistas.

8.6 Favoritos

Los usuarios pueden marcar recetas como favoritas. El botón funciona como toggle. Existe un listado dedicado (`/favorites/`) con todas las recetas marcadas.

8.7 Notificaciones

Se generan notificaciones automáticas cuando otro usuario forkea o comenta en una receta propia. El badge en la barra de navegación muestra el contador de notificaciones no leídas. Al visitar la bandeja se marcan como leídas.

8.8 Subida de imágenes

Las recetas admiten imagen de dos formas: URL externa o subida de archivo. Los archivos subidos se guardan en `static/uploads/` con nombre UUID. Se limita a 5 MB y formatos jpg, png, gif, webp.

8.9 Exportación a Markdown

Cualquier receta puede descargarse en formato Markdown (`.md`) con título, descripción, autor, etiquetas, ingredientes y pasos formateados.

8.10 Búsqueda y filtrado

El índice permite buscar recetas por texto (título o descripción), filtrar por etiqueta o por autor. Los filtros se aplican mediante parámetros GET.

Árbol de Versiones (Forks) - Recetario Fork

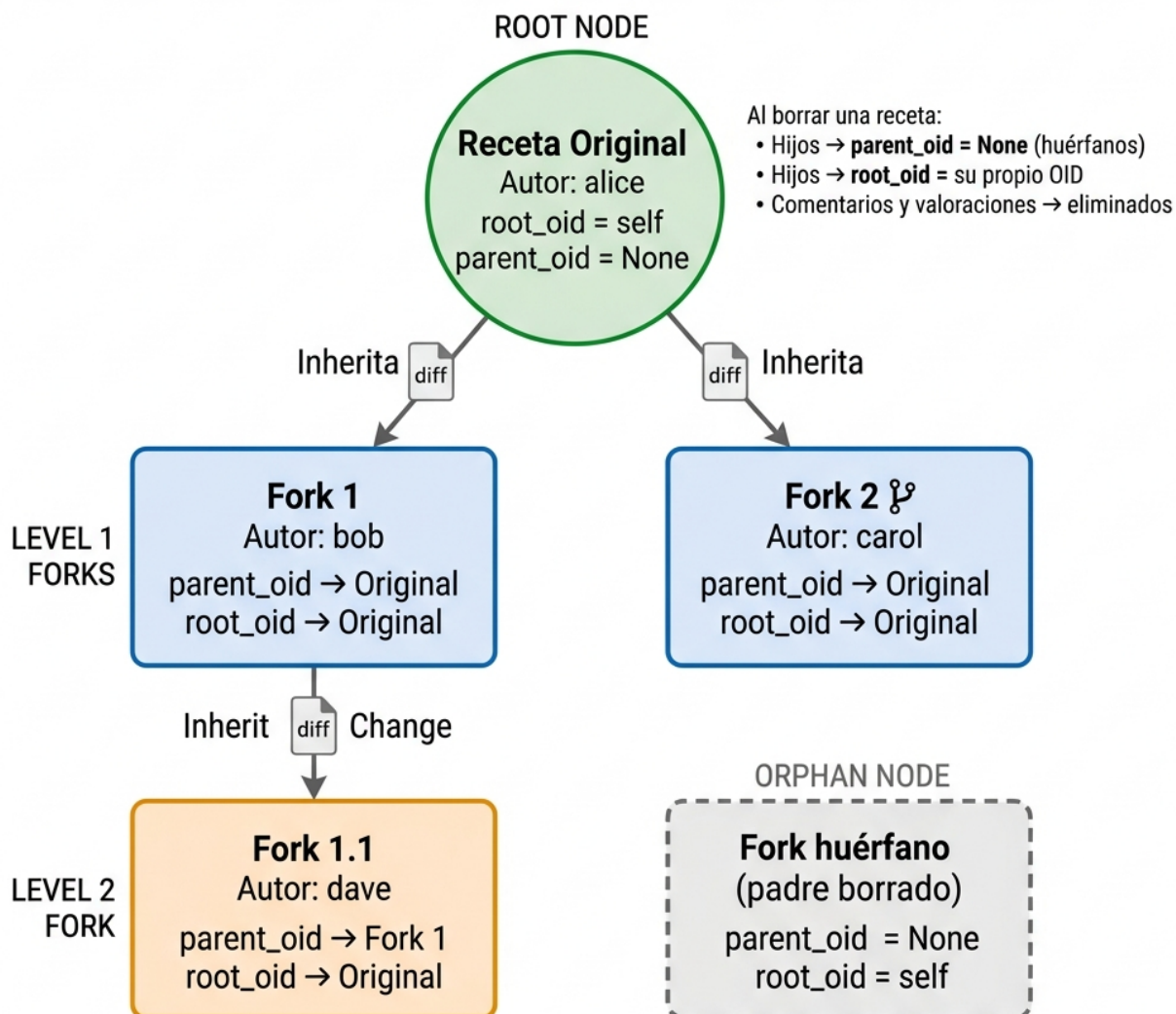


Figura 4 - Ejemplo de árbol de forks entre recetas

9. Diagramas

A continuación se presentan los diagramas de diseño de la aplicación. Estos diagramas se encuentran también como archivos PNG independientes en el directorio docs/diagrams/.

9.1 Diagrama de arquitectura

Arquitectura del Sistema - Recetario Fork

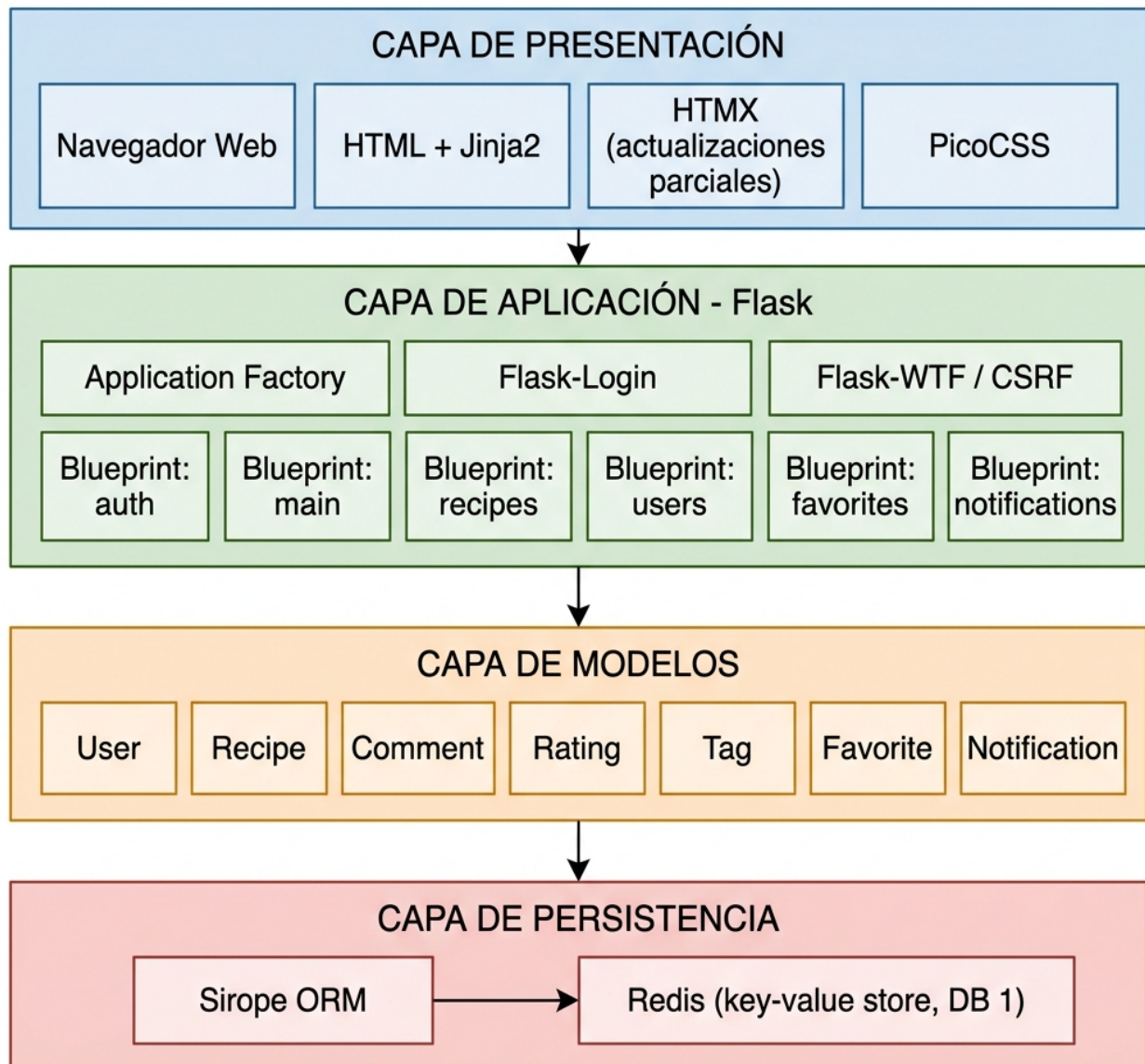


Figura 5 - Arquitectura general de la aplicación

9.2 Diagrama de clases

Recetario Fork

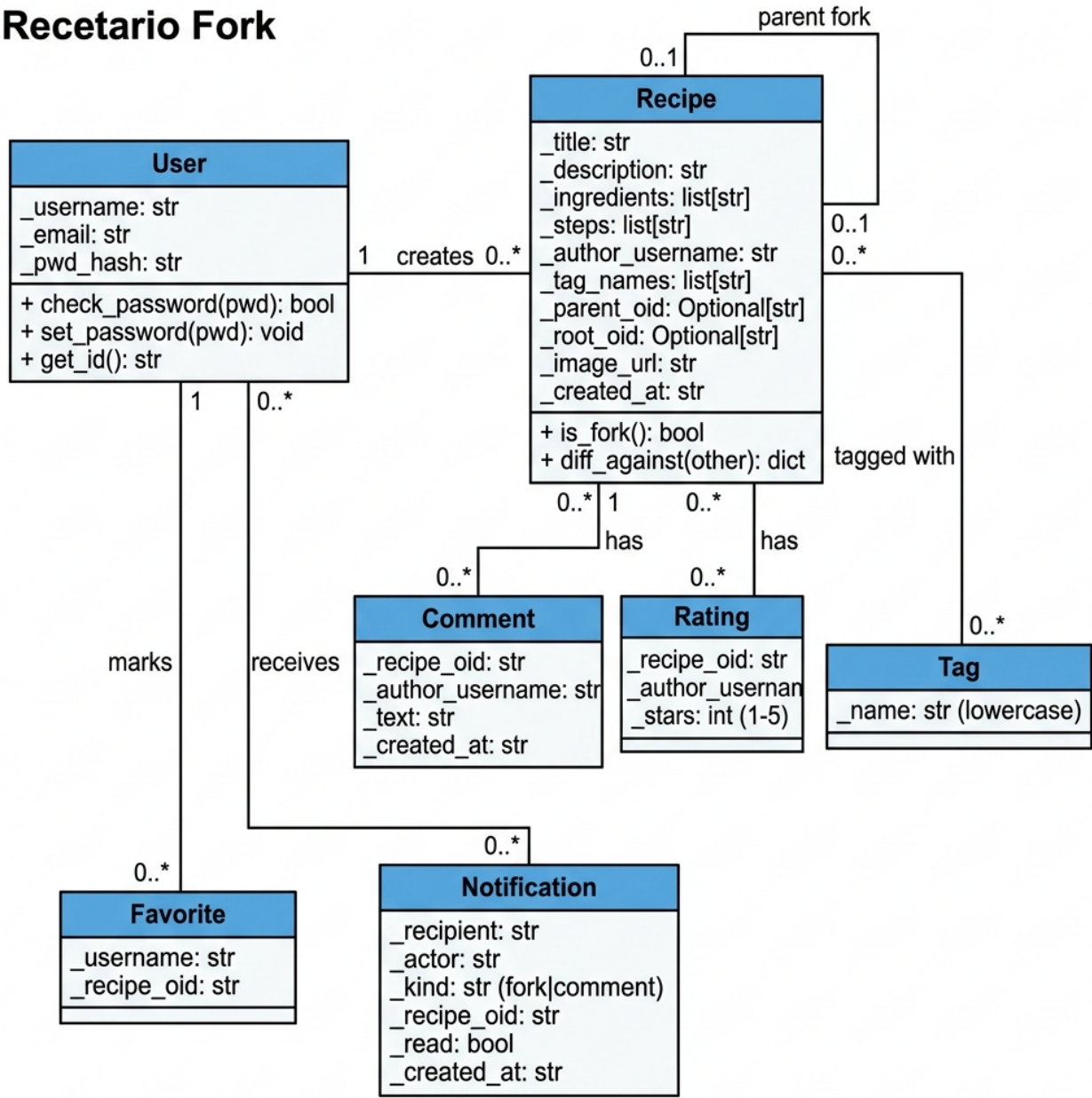


Figura 6 - Diagrama de clases del modelo de dominio

9.3 Diagrama de secuencia

Diagrama de Secuencia: Crear un Fork de Receta

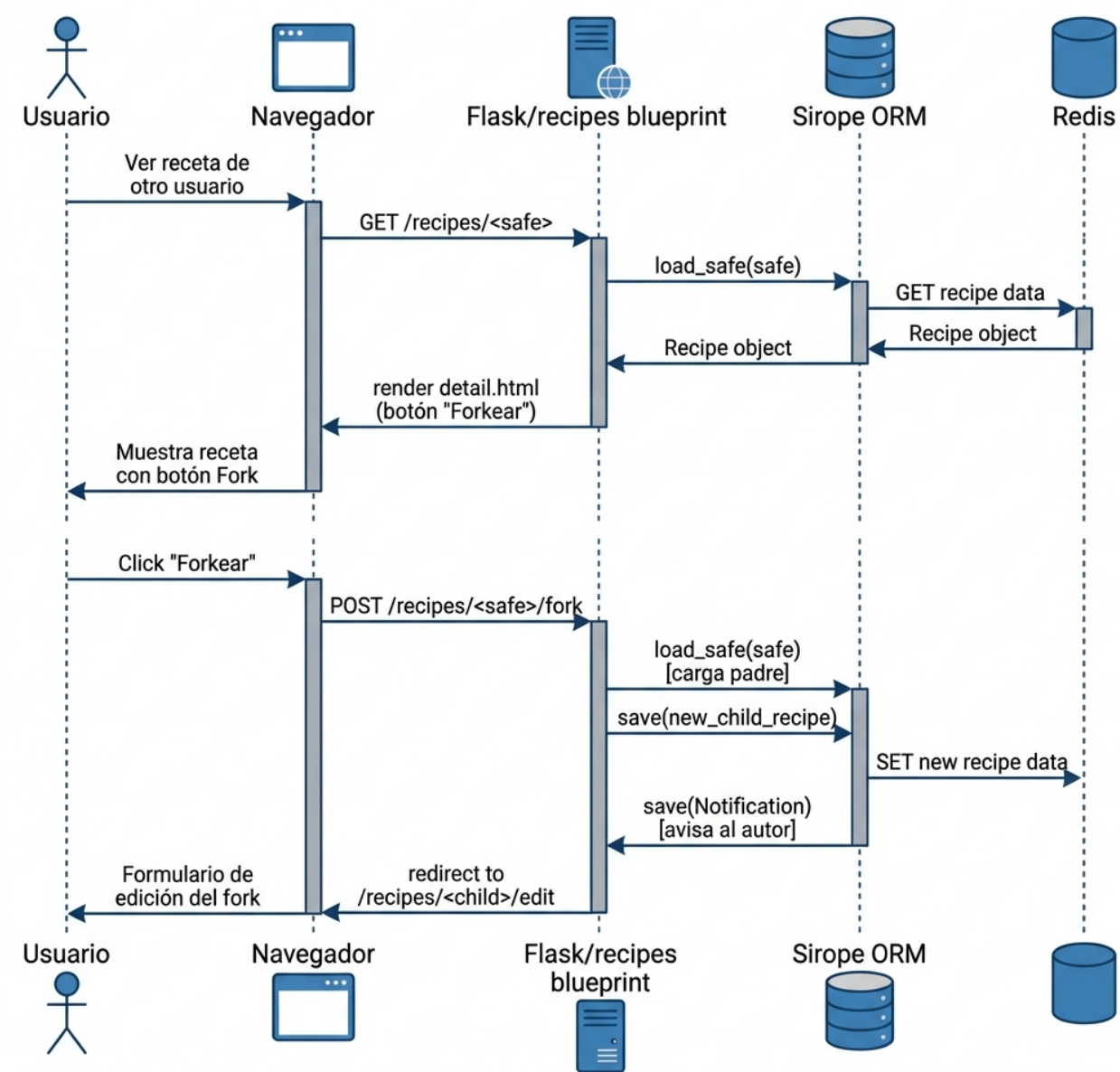


Figura 7 - Diagrama de secuencia: flujo de fork de una receta

9.4 Mapa de rutas URL

Mapa de Rutas URL - Recetario Fork

METHOD	URL	BLUEPRINT	DESCRIPCIÓN
GET	/	main	Listado de recetas (con filtros q, tag, author)
GET/POST	/auth/login	auth	Inicio de sesión
GET/POST	/auth/register	auth	Registro de usuario
GET	/auth/logout	auth	Cerrar sesión
GET/POST	/recipes/new	recipes	Crear receta nueva
GET	/recipes/<safe>	recipes	Detalle de receta
GET	/recipes/<safe>	recipes	Editar receta
GET/POST	/recipes/<safe>/edit	recipes	Editar receta
POST	/recipes/<safe>/delete	recipes	Borrar receta
POST	/recipes/<safe>/fork	recipes	Forkear receta
GET	/recipes/<safe>/tree	recipes	Árbol de versiones
GET	/recipes/<safe>/diff	recipes	Diferencias con padre
POST	/recipes/<safe>/comment	recipes	Añadir comentario
POST	/recipes/<safe>/comment/<csafe>/delete	recipes	Borrar comentario
POST	/recipes/<safe>/rate	recipes	Valorar receta
POST	/recipes/<safe>/favorite	recipes	Toggle favorito
POST	/recipes/<safe>/merge-from-parent	recipes	Merge del padre
GET	/recipes/<safe>/export	recipes	Exportar Markdown
GET	/user/<username>	users	Perfil de usuario
GET	/favorites/	favorites	Mis favoritos
GET	/notifications/	notifications	Bandeja de notificaciones
POST	/notifications/clear	notifications	Limpiar notificaciones

Figura 8 - Mapa completo de rutas URL de la aplicación

9.5 Árbol de forks

Árbol de Versiones (Forks) - Recetario Fork

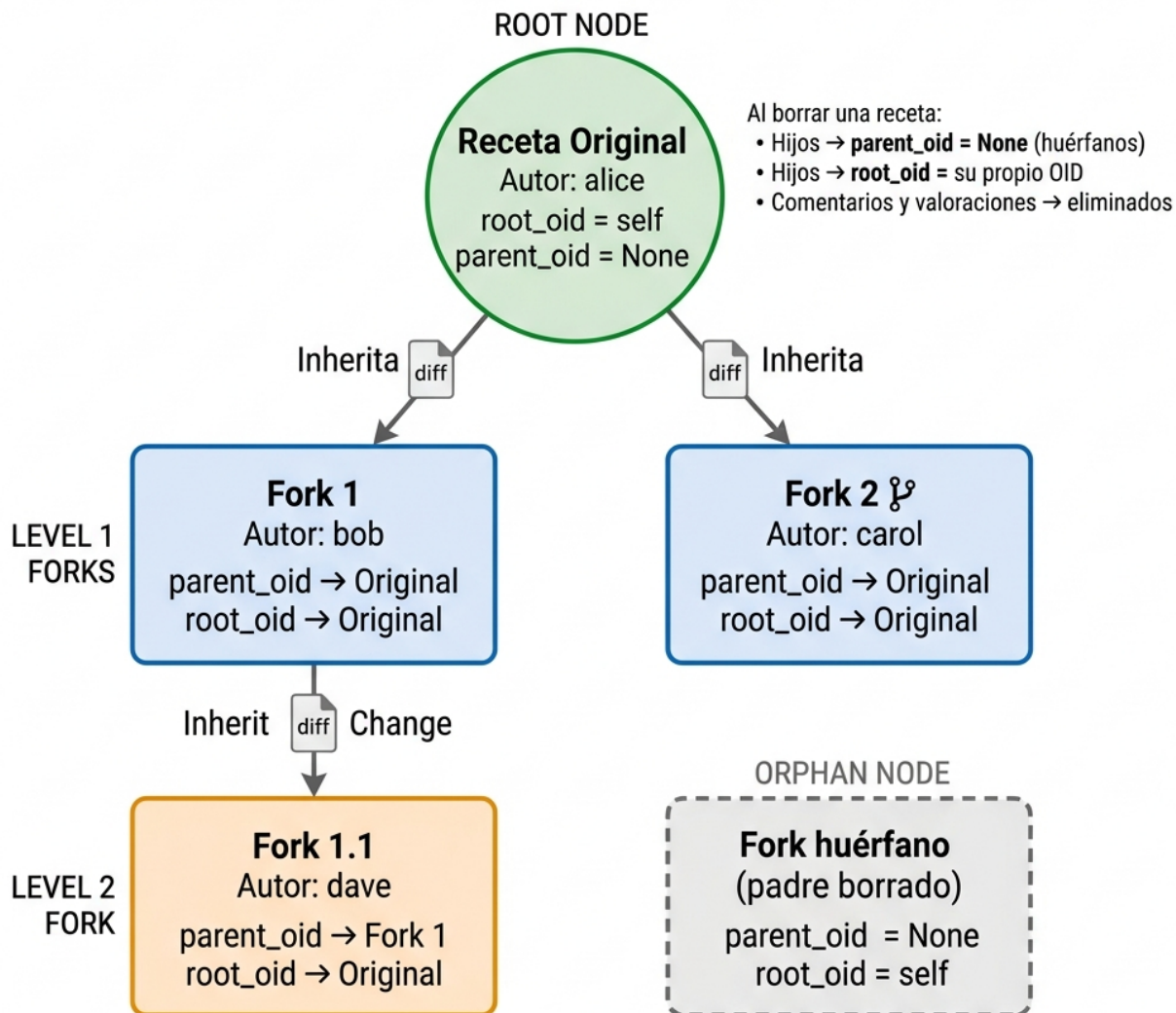


Figura 9 - Ejemplo de árbol de forks entre recetas

10. Estructura de ficheros

```
src/
??? run.py                # Punto de entrada
??? config.py             # Clase Config (SECRET_KEY, UPLOAD_FOLDER, etc.)
??? requirements.txt      # Dependencias Python
??? README.md            # Documentación rápida
??? docs/
?   ??? generate_pdf.py   # Generador de esta documentación
?   ??? diagrams/        # Diagramas PNG
?       ??? architecture_diagram.png
?       ??? class_diagram.png
?       ??? fork_tree_diagram.png
?       ??? sequence_diagram.png
?       ??? url_routes_diagram.png
??? app/
    ??? __init__.py       # Application factory (create_app)
    ??? extensions.py     # Sirope, LoginManager, helpers OID
    ??? errors.py         # Handlers 404 / 500
    ??? forms.py          # WTForms (Login, Register, Recipe, Comment, Rating)
    ??? models/
        ??? __init__.py   # Re-exporta todas las entidades
        ??? user.py       # User (UserMixin)
        ??? recipe.py     # Recipe (forks, diff)
        ??? comment.py    # Comment
        ??? rating.py     # Rating (1-5 estrellas)
        ??? tag.py        # Tag
        ??? favorite.py   # Favorite
        ??? notification.py # Notification (fork/comment)
    ??? views/
        ??? __init__.py   # Blueprint auth
        ??? auth.py       # Blueprint main (índice)
        ??? main.py       # Blueprint recipes (CRUD + fork + diff)
        ??? recipes.py     # Blueprint users (perfil)
        ??? users.py       # Blueprint favorites
        ??? favorites.py   # Blueprint notifications
    ??? templates/
        ??? base.html     # Layout base
        ??? index.html    # Página principal
        ??? error.html    # Página de error genérica
        ??? auth/login.html, register.html
        ??? recipes/detail.html, form.html, tree.html, diff.html
        ??? recipes/_card.html, _comments.html, _rating.html, _fav_btn.html
        ??? users/profile.html
        ??? favorites/list.html
        ??? notifications/inbox.html
    ??? static/
        ??? style.css     # Estilos personalizados
        ??? uploads/      # Imágenes subidas por usuarios
```

11. Instalación y ejecución

11.1 Clonar e instalar dependencias

```
cd src
python -m venv .venv
. .venv/Scripts/activate      # Windows (bash)
# .venv\Scripts\activate      # Windows (cmd)
# source .venv/bin/activate    # Linux/Mac
pip install -r requirements.txt
```

11.2 Arrancar Redis

```
redis-server &
```

O en Windows, iniciar Redis como servicio o ejecutar redis-server.exe.

11.3 Ejecutar la aplicación

```
# Opción 1:
python run.py

# Opción 2:
set FLASK_APP=run.py      # Windows cmd
flask run
```

La aplicación queda disponible en <http://127.0.0.1:5000>.

11.4 Variables de entorno opcionales

Variable	Descripción
SECRET_KEY	Clave de sesión Flask (defecto: dev-change-me)
REDIS_HOST	Host de Redis (defecto: localhost)
REDIS_PORT	Puerto de Redis (defecto: 6379)
REDIS_PASSWORD	Contraseña de Redis (defecto: ninguna)
REDIS_DB	Base de datos Redis (defecto: 1)

12. Decisiones de diseño

12.1 OIDs seguros en URLs

Los OID internos de Sirope se convierten a su forma URL-safe mediante `srp.safe_from_oid()` antes de ser enviados al navegador. Esto evita problemas con caracteres especiales en las URLs y facilita la interoperabilidad.

12.2 Política de borrado y huérfanos

Al borrar una receta, sus forks hijos no se eliminan: se convierten en huérfanos (`parent_oid = None`) y pasan a ser raíz de su propio subárbol (`root_oid = su propio OID`). Esto preserva el trabajo de otros usuarios. Los comentarios, valoraciones, favoritos y notificaciones asociados a la receta borrada sí se eliminan.

12.3 Unicidad de valoraciones

No existe una restricción a nivel de modelo para impedir ratings duplicados. La unicidad se garantiza en la capa de vistas: al votar, se busca un Rating existente del mismo usuario para la misma receta. Si existe, se actualizan las estrellas; si no, se crea uno nuevo.

12.4 Hash de contraseñas

Las contraseñas se almacenan de forma segura utilizando la librería `werkzeug.security`, que emplea algoritmos modernos y robustos (`scrypt` o `pbkdf2`) con generacion automatica de salt. Esto protege a los usuarios contra ataques de fuerza bruta y diccionarios.

12.5 Context processor para notificaciones

El contador de notificaciones no leídas se inyecta en todas las plantillas mediante un `context_processor` registrado en `create_app()`. Esto permite que el badge del icono de la campanilla se actualice en cada carga de página.

12.6 Subida de imágenes

Se implementan dos vías para asociar imagen a una receta: pegar una URL externa o subir un archivo. El archivo tiene prioridad si ambos se proporcionan. Los archivos se guardan con nombre UUID para evitar colisiones. El tamaño máximo es de 5 MB (`MAX_CONTENT_LENGTH` en `config.py`).

12.7 HTMX para actualizaciones parciales

Las vistas que reciben peticiones HTMX (detectadas via el header `HX-Request`) devuelven solo el fragmento HTML necesario en lugar de la pagina completa. Esto proporciona una experiencia fluida sin necesidad de escribir JavaScript personalizado.

12.8 Proteccion global CSRF

Toda la aplicacion esta protegida contra vulnerabilidades Cross-Site Request Forgery (CSRF) mediante la extension `Flask-WTF CSRFProtect`. Esto exige que tanto los formularios HTML tradicionales como las llamadas asincronas de HTMX incluyan el token de seguridad.